

Core Concepts — Tracing & the LangSmith Data Model

Track 16.5 — 🦋 LangSmith · 16-ai-tech-stack/tracks/langsmith/notes/01-core-concepts.md

1. LLM apps break differently, and your existing tools cannot see it

You already know how to operate software. You have logs, metrics, traces, alerts, a pager. None of it fires when an LLM application breaks, because LLM applications mostly break **while returning HTTP 200**.

Normal software	LLM application
Crashes. Stack trace. Line number.	Returns a fluent, confident, wrong paragraph.
Deterministic: same input → same output	Same input → different output, run to run
Regression = a failing test	Regression = "answers feel worse this week"
Cost is ~fixed per request	Cost varies 50x with context length, and drifts silently
Latency = your code	Latency = someone else's GPU queue
Bug is in <i>your</i> code	Bug is in a prompt, a retrieved chunk, a tool schema, or the model itself

Four failure modes have no analogue in normal operations:

1. **Silent quality regression.** Someone edits three words in a system prompt. Nothing errors. Accuracy on one customer segment drops 12%. You find out from a support ticket six weeks later.
2. **Cost drift.** A retriever's `k` goes from 4 to 8. Every request now carries 2x the context. Latency rises 300ms — under your alert threshold — and the bill rises 90%, which nobody is paging on.
3. **Nondeterminism.** The bug reproduces one time in seven. Without a recording of the *exact* failing execution, you are debugging from memory.
4. **No stack trace.** "The answer got worse" points at nothing. Was it retrieval? The prompt? A tool that returned an empty list and got silently stringified into the context? Model provider changed something?

The answer to all four is the same artifact: **a trace**. Not a log line — a structured recording of the whole execution, with every input, every output, every intermediate step, timing, and token counts.

Frame it this way: in normal software the log is a *supplement* to the stack trace. In LLM software the trace *is* the stack trace. It is not observability nice-to-have; it is the only debugger you get.

2. The data model — six nouns

LangSmith is, underneath the UI, a tree database with an opinion. Learn these six nouns and the product stops being mysterious. All of them are reimplemented in `../code/minimal_tracer.py`, which runs offline.

```
PROJECT "support-agent-prod"          a container for traces from one app/service
|
+-- TRACE (one top-level operation, e.g. one user question)
|   |
|   +-- RUN support_agent [chain] 1,240ms <- the root run
|       |                        trace_id == its own id
|       +-- RUN retrieve [retriever] 84ms
|       +-- RUN build_prompt [prompt] 0ms
|       +-- RUN chat_completion [llm] 910ms 1,150 tok $0.0041
|       +-- RUN check_policy [tool] 31ms ERROR ValueError
|
+-- THREAD "session-8812" groups SIBLING traces into one conversation
```

- **Run** — one unit of work: an LLM call, a retrieval, a tool invocation, a parse. Carries `inputs`, `outputs`, `error`, `start_time`, `end_time`, `run_type`, `tags`, `metadata`, and (for `llm` runs) token usage. A run is the atom. Everything else is an aggregation of runs.
- **Trace** — the tree of runs for a single top-level operation. Every run in it shares a `trace_id`. (LangSmith caps a single trace at 25,000 runs; if you hit that, your agent has a loop bug.)
- **run_type** — `llm`, `chain`, `tool`, `retriever`, `prompt`, `parser`. This is not cosmetic: it tells the backend to render token usage and message roles for `llm` runs, documents for `retriever` runs, and arguments/returns for `tool` runs. Get it wrong and the UI shows you a blob of JSON.
- **Project** (a.k.a. tracing project) — the container. One per app per environment. `support-agent-prod`, `support-agent-staging`. Not one per team, not one per model.
- **Thread** — a `thread_id` in metadata that groups *sibling traces* into one multi-turn session. This is the single most commonly skipped field and the one you will most regret skipping, because multi-turn bugs ("it forgot the order number from turn 2") are invisible one trace at a time.
- **Feedback** — a score attached to a run by id: continuous or categorical, from a human, a user thumbs-up, or an automated evaluator. Feedback is what makes traces *searchable by quality* instead of by text.

Two more that are technically metadata but behave like first-class citizens:

- **Tags** — flat strings for filtering (`prod`, `canary`, `experiment-7`).
- **Metadata** — key-value pairs. Put your **git SHA** / **release version** here on day one. Without it, "when did this start" is unanswerable, and "when did this start" is the first question every time.

3. Instrumenting code — four mechanisms, in order of effort

3.1 Wrap the provider client (5 seconds, highest value)

```
from langsmith.wrappers import wrap_openai # also wrap_anthropic
client = wrap_openai(openai.OpenAI())
```

Every call now produces an `llm` run with model, messages, tool calls, token usage and cost, rendered properly. This one line does more than the next twenty.

3.2 @traceable on your own functions

```
from langsmith import traceable

@traceable(run_type="retriever")
def retrieve(question: str) -> list[str]: ...

@traceable(name="support_agent", tags=["prod"], metadata={"release": GIT_SHA})
def support_agent(question: str) -> str: ...
```

Nesting is automatic — the decorator tracks the current run in a `contextvar`, so anything called inside a traced function becomes its child with no plumbing. That mechanism (and its limits) is exactly what `minimal_tracer.py` rebuilds.

3.3 Automatic, if you use LangChain or LangGraph

Set `LANGSMITH_TRACING=true` and LangChain/LangGraph emit full traces with zero imports. Nodes, edges, state transitions and tool calls all appear. That is the honest upside of staying inside the ecosystem (see `../../langgraph/` and `../../langchain/`).

3.4 Manual run trees, for the awkward cases

```
from langsmith.run_helpers import trace

with trace(name="batch_scoring", run_type="chain", inputs={"n": len(batch)}) as rt:
    result = score_all(batch)
    rt.end(outputs={"mean": result.mean})
```

You need this when the `contextvar` cannot follow your code: streaming generators, callback-driven frameworks, `multiprocessing` pools, or a job that resumes in a different worker. If spans are showing up as orphaned roots instead of children, this is almost always why.

3.5 Environment variables — and the bug everyone ships once

```
export LANGSMITH_TRACING=true                # THE MASTER SWITCH
export LANGSMITH_API_KEY="lsv2_pt..."
export LANGSMITH_PROJECT="support-agent-prod"
export LANGSMITH_TRACING_SAMPLING_RATE=0.1   # 0.0-1.0
# export LANGSMITH_ENDPOINT="https://eu.api.smith.langchain.com" # EU region
```

The bug: you set `LANGSMITH_API_KEY` and forget `LANGSMITH_TRACING`. Nothing raises. `@traceable` degrades to a plain function call. No traces appear, and there is no error to search for. If your traces are missing, check this variable before you check anything else.

Tracing is designed to be non-blocking — the SDK batches and ships runs in the background, so a slow or unreachable backend should not stall your request path. Don't treat that as a licence to skip a timeout budget on your own critical path.

3.6 Sampling and PII — the two decisions to make before production

Sampling. `LANGSMITH_TRACING_SAMPLING_RATE=0.1` keeps 10% of traces. Sample at the **root**, never per-span, or you get trees with holes where the interesting child used to be. A sane default: sample healthy traffic at 1–10%, and keep 100% of anything with an error or negative feedback — which is a rule on the backend, not a client-side decision.

PII. You are shipping user text to a third-party SaaS. Three seams, from surgical to blunt:

Mechanism	Scope	Use when
<code>process_inputs= / process_outputs= on @traceable</code>	one function	a specific function touches SSNs
<code>Client(anonymizer=create_anonymizer([...]))</code>	whole client, regex rules	emails/cards/phones everywhere
<code>Client(hide_inputs=fn, hide_outputs=fn)</code>	whole client, callable	structure-aware masking
<code>LANGSMITH_HIDE_INPUTS=true</code>	everything	regulated data, no exceptions granted

Redact at the **SDK boundary**, inside your process. Server-side scrubbing is theatre: by the time the backend can scrub it, the data has already crossed a network boundary you don't own. Note the trade: with inputs hidden you keep shape, latency, cost and error rate, and lose most of the debugging value. Make that call deliberately, with `../../../../12-ai-governance/` and `../../../../13-ai-security/` in the room — and write down who decided.

4. Reading a trace — what to look at, in order

You have a trace of a bad answer open. Do this, in this order:

1. **Look at the tree shape first, not the text.** Is there a node you did not expect? A tool called twice? A retriever that returned zero documents? Shape bugs are the cheapest to find and the most common.
 2. **Read the retrieved context, not the answer.** In RAG systems the majority of "hallucinations" are the model faithfully summarising the wrong chunks. See `../../../../08-rag/`.
 3. **Read the *final* prompt.** Not your template — the rendered string, with every variable filled. The number of bugs that are "the variable was empty and the f-string cheerfully rendered `None`" is remarkable.
 4. **Check the latency waterfall.** One child usually owns 80%+ of the wall time. That is your optimisation target and nothing else is.
 5. **Check tokens on the way in.** Sudden context growth is the tell for prompt-template bloat, a retriever `k` change, or unbounded conversation history.
 6. **Only now** argue about the model.
-

5. Cost and latency are first-class, not an afterthought

Cost is a **derived field**: $(\text{prompt_tokens} \times \text{input_rate}) + (\text{completion_tokens} \times \text{output_rate})$, computed per `llm` run and rolled up the tree. The rollup is what matters — a `chain` run has no cost of its own, but the subtree under it does, and "which endpoint burns the budget" is a rollup query.

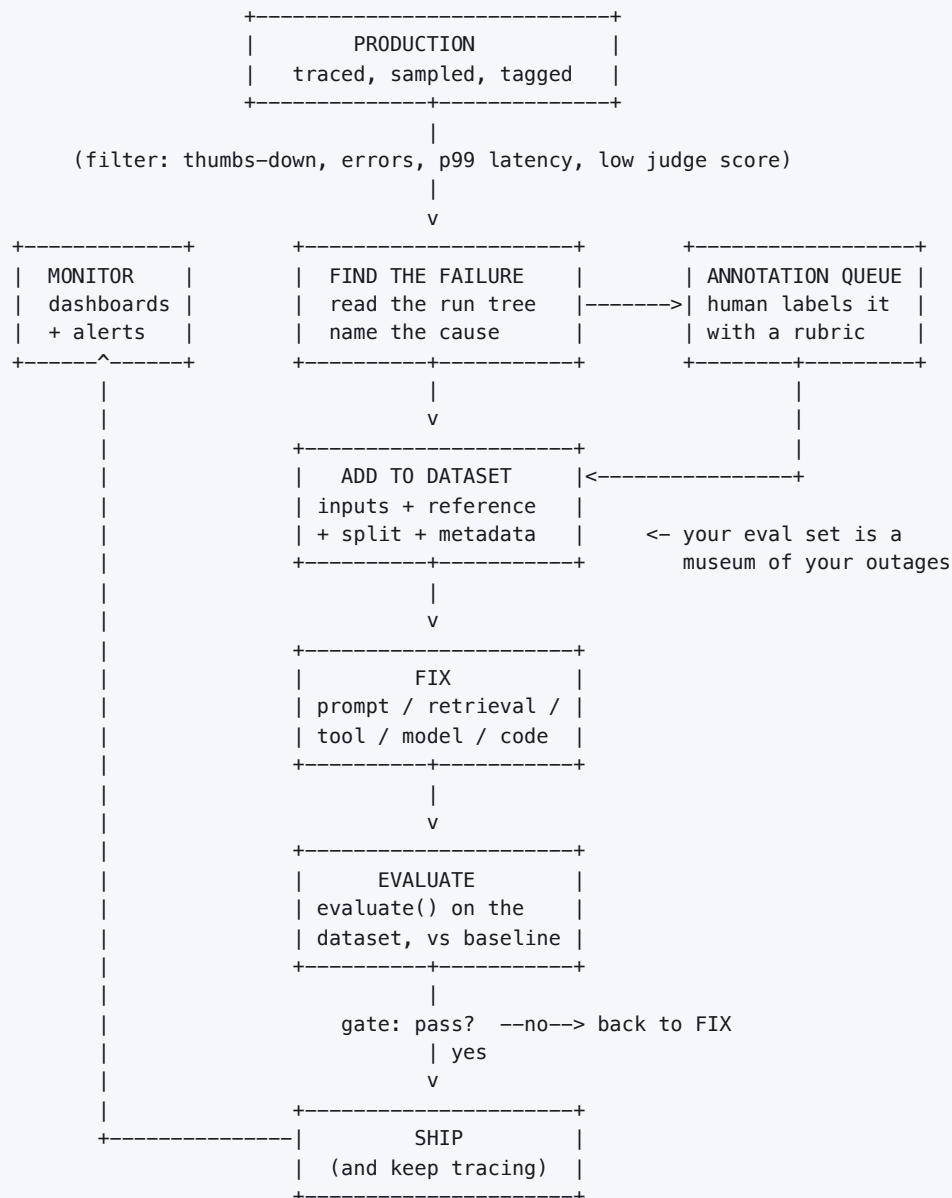
What to actually alert on, roughly in order of how often it saves you:

Signal	Why	A starting threshold
p95 cost per trace	catches prompt bloat and retriever changes	+25% week over week
p95 latency	users leave; means hide the tail	your SLO, minus headroom
Error-run rate	tool timeouts, schema failures, rate limits	>1% of spans
Negative feedback rate	the only direct quality signal you get free	>2x trailing baseline
Trace volume	a drop means your instrumentation broke	+/-50% day over day

The classic LLM incident is not a 500. It is a prompt change that tripled context length and went unnoticed for nine days, at which point Finance noticed before Engineering did.

6. The operating loop

This is the whole discipline. Everything else in this track is an implementation detail of this diagram.



Three things people get wrong about this loop:

- **They start at "evaluate".** They write an eval set in a meeting, before they have a single trace. The set reflects what the team imagined users would ask, which is never what users ask. Start at *production* (or at manual traces from your own dogfooding), and harvest.
- **They stop at "ship".** The loop is a loop. The monitor step feeds the next iteration or you are just doing QA with extra steps.
- **They skip the human.** Every automated judge in the world is calibrated against human labels, or it is calibrated against nothing. See §4 of `02-evaluation-and-operations.md`.

7. What LangSmith is *not*

Be precise about the boundaries, because vendors are not:

- **Not LangChain-only.** `@traceable` and the client wrappers work on raw SDK calls, FastAPI handlers, DSPy, your own agent loop. LangSmith is a tracing backend that happens to be built by the LangChain team. The framework coupling is a marketing impression, not a technical constraint. (It *does* get automatic tracing for LangChain/LangGraph for free, which is a real advantage, not a lock-in.)
- **Not an APM.** It will not tell you about your database connection pool. Run it alongside Datadog / Grafana, not instead. It can speak OTLP in both directions, so it can sit inside an existing stack.
- **Not a guardrail.** It observes; it does not block. Blocking policy lives in `../../../../13-ai-security/`.
- **Not free at scale.** Trace volume is the billing unit. This is why sampling is a design decision, not a tuning knob. Self-hosting and BYOC exist but are Enterprise-plan gated — budget for that conversation early if you are in a regulated shop.

8. Where this returns

Idea here	Where it returns
The run tree as the primary debugging artifact	<code>../../../../langgraph/</code> — every node is a span
Retrieved context is the usual culprit	<code>../../../../08-rag/</code> — chunking and reranking
Tool-call spans and agent loops	<code>../../../../06-ai-agents/</code> , <code>../../../../07-agentic-ai/</code>
Datasets, judges, pairwise, contamination	<code>../../../../15-ai-evals/</code> — the theory for all of it
PII redaction at the SDK boundary	<code>../../../../13-ai-security/</code>
Who decided to log user text to a vendor	<code>../../../../12-ai-governance/</code>
Traces and eval runs as audit evidence	<code>../../../../14-ai-compliance/</code>
Cost per trace -> cost per model choice	<code>../../../../11-llm-models/</code>
Where observability sits in the stack	<code>../../../../10-ai-architecture/</code>